

Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales

Carrera de Informática



Rastreo Global de Vuelos y Logística en Tiempo Real

Nombres:

- Galarza Salguero Andres William
- Rojas Cerda Luis Alejandro

Docente:

Celia Elena Tarquino Peralta

La Paz, Bolivia

1. *Introducción*

El proyecto "Sistema Global de Rastreo de Vuelos y Optimización Logística" es una plataforma de alta disponibilidad diseñada para la monitorización en tiempo real de flotas aéreas y la gestión de cadenas de suministro transcontinentales. En un ecosistema donde cada segundo de retraso se traduce en pérdidas millonarias, esta solución emplea una arquitectura híbrida de base de datos para maximizar la eficiencia.

La aplicación utiliza Redis como motor de mensajería y almacenamiento en memoria para gestionar la telemetría inmediata (coordenadas GPS, altitud, velocidad) mediante estructuras de datos geoespaciales. Por otro lado, delega en MongoDB la gestión de documentos complejos, como el historial detallado de manifiestos de carga, registros de mantenimiento y análisis de rutas históricas. Esta combinación permite no solo visualizar dónde está un paquete ahora, sino predecir tiempos de entrega basados en grandes volúmenes de datos previos.

2. *Problemática*

La logística aérea moderna enfrenta desafíos críticos que los sistemas tradicionales de bases de datos relacionales no logran resolver con agilidad:

- **Latencia en Datos Volátiles:** Los sistemas convencionales sufren cuellos de botella al intentar procesar miles de actualizaciones de ubicación por segundo de múltiples aeronaves simultáneamente.
- **Inconsistencia en el Rastreo:** La falta de un sistema de "caché" inteligente provoca que la última ubicación conocida se pierda o sea lenta de recuperar si el sistema principal está bajo carga pesada.
- **Rigidez de Esquemas:** La información de logística aérea es heterogénea (datos meteorológicos, especificaciones de carga, rutas cambiantes). Las bases de datos rígidas dificultan la integración de nuevos tipos de sensores o metadatos sin detener la operación.
- **Escalabilidad ante Eventos Críticos:** En situaciones de crisis (clima adverso o cierres de aeropuertos), el volumen de consultas y actualizaciones se dispara, colapsando los sistemas que no están diseñados para una escalabilidad horizontal masiva.

Enfoque de Solución con Mongo y Redis

Para mitigar esto, el proyecto propone:

Capa de Tiempo Real (Redis): Reducir la latencia a milisegundos para el rastreo activo.

Capa de Persistencia (MongoDB): Almacenar documentos flexibles que permitan una trazabilidad completa del ciclo de vida de la carga sin las limitaciones de un esquema fijo.

3. *Objetivos*

3.1. *Objetivo General*

Desarrollar un sistema de monitoreo global de alta disponibilidad que utilice una arquitectura híbrida de bases de datos (NoSQL y Cache) para procesar telemetría de vuelos y activos logísticos en tiempo real, garantizando latencias mínimas en la consulta y una persistencia escalable de datos históricos.

3.2. *Objetivos Específicos*

1. **Optimización de Latencia con Redis:** Implementar una capa de almacenamiento en memoria con Redis para gestionar el estado "vivo" de miles de aeronaves, permitiendo actualizaciones de posición con latencia inferior a los 10ms para los usuarios finales.
2. **Persistencia Escalable en MongoDB:** Estructurar una base de datos documental en MongoDB para el almacenamiento masivo de registros históricos (logs de vuelo), aprovechando su flexibilidad de esquema para integrar diversos sensores IoT y manifiestos de carga.
3. **Sincronización Eficiente de Datos:** Establecer un flujo de datos que sincronice los eventos críticos almacenados en Redis hacia MongoDB, asegurando que la información operativa esté disponible para auditorías y análisis de largo plazo sin penalizar el rendimiento del sistema en vivo.
4. **Consultas Geoespaciales Avanzadas:** Utilizar los índices geoespaciales nativos de **MongoDB** para realizar cálculos de proximidad, geofencing y optimización de rutas logísticas basados en coordenadas globales.
5. **Gestión de Mensajería en Tiempo Real:** Aprovechar las capacidades de Pub/Sub de Redis para distribuir alertas instantáneas sobre cambios de estado en los vuelos (despegues, aterrizajes, emergencias) a múltiples clientes conectados simultáneamente.

4. *Contexto de Desarrollo*

4.1. *Tipo de Aplicación*

El sistema se define como una Aplicación Web de Monitoreo de Datos Masivos en Tiempo Real con arquitectura distribuida. Es una herramienta de carácter híbrido:

Back-end de alto rendimiento: Para la ingesta constante de datos desde APIs externas (como OpenSky).

Front-end reactivo: Para la visualización dinámica de activos sobre mapas globales sin necesidad de recargar la página.

Sistema de Gestión de Datos Híbrido: Que separa el estado efímero (posiciones actuales) de la persistencia histórica (análisis logístico).

4.2. Stack Tecnológico

La arquitectura se fundamenta en tecnologías orientadas a la escalabilidad y baja latencia:

- **Base de Datos : MongoDB.** Utilizada como base de datos documental para almacenar el histórico de vuelos, metadatos de aeronaves, planes de logística y registros de usuarios.
- **Base de Datos : Redis.** Implementada para gestionar el estado actual de los vuelos ("Last Known Position") y como broker de mensajes para la actualización inmediata de la UI.
- **Lenguaje de Programación: Python** (aprovechando librerías para manejo de datos y conexión con OpenSky).
- **Framework de API: Flask / FastAPI.** Encargado de la lógica de negocio y la exposición de endpoints para el cliente.
- **Entorno de Ejecución: Docker.** Para asegurar la portabilidad del stack (especialmente para los contenedores de Redis y MongoDB) en diferentes entornos.

4.3. Herramientas de Desarrollo

Se utilizan herramientas de grado profesional para garantizar la calidad del software y el flujo de trabajo:

- **Sistema Operativo: Debian 13 (Trixie) y Windows 10.** Proporcionan un entorno estable y robusto para la ejecución de servicios de servidor y gestión de paquetes.
- **Documentación de Arquitectura:** Herramientas de modelado UML para la creación de diagramas de casos de uso, clases y secuencia siguiendo la metodología de Larman.

- **Pruebas y Testing: Pytest** para pruebas unitarias y generación de casos de prueba automatizados para validar los algoritmos de logística.
- **Cliente de Base de Datos: Compass** (para MongoDB) y **Redis Insight** para la visualización y depuración de los datos almacenados.

5. Marco Teórico y Metodológico

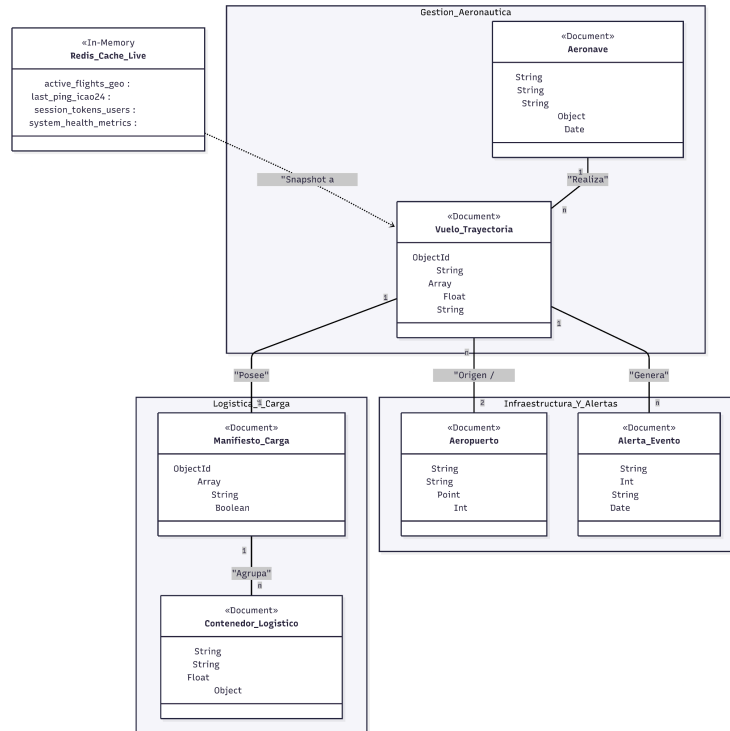
5.1. Bases de Datos NoSQL

A diferencia de los modelos relacionales tradicionales, las bases de datos NoSQL permiten el almacenamiento de datos no estructurados o semi-estructura

dos con alta escalabilidad horizontal. En este proyecto se aplica un enfoque políglota:

Modelo Documental (MongoDB): Permite almacenar la información de vuelos y logística como documentos JSON/BSON, facilitando la evolución del esquema sin tiempos de inactividad.

Modelo Clave-Valor (Redis): Utilizado para operaciones que requieren latencia de microsegundos, ideal para estados de alta volatilidad como las coordenadas actuales de una aeronave.



5.2. Domain-Driven Design (DDD)

Se aplica DDD para segmentar la complejidad del rastreo global y la logística, asegurando que el software refleje fielmente las reglas de negocio aeronáuticas.

5.2.1. Entidades

Son objetos con una identidad única que no cambia, aunque sus atributos sí lo hagan.

- **Aeronave:** Identificada por su código único **icao24**. Su estado (altitud, velocidad) cambia constantemente, pero la entidad física es la misma.
- **Contenedor Logístico:** Identificado por su **tracking_id**. Es el objeto central de seguimiento en la cadena de suministro.

5.2.2. Agregados

Grupos de objetos relacionados que se mantienen como una unidad de consistencia.

- **Agregado de Operación de Vuelo:** La raíz es el **Vuelo**. Contiene la **Trayectoria** y las **Alertas** generadas. Cualquier cambio en la trayectoria se gestiona a través de la raíz del vuelo.

- **Agregado de Carga:** La raíz es el **Manifiesto de Carga**, que encapsula múltiples **Contenedores**. Esto asegura que si el manifiesto se cancela, todos los contenedores asociados actualicen su estado en bloque.

5.2.3. Servicios de Dominio

Lógica que no encaja en una sola entidad y coordina múltiples objetos.

- **Servicio de Sincronización Real-Time:** Coordina el paso de datos desde las llaves rápidas de Redis hacia los documentos históricos de MongoDB.
- **Calculador de Geofencing:** Determina si una aeronave ha ingresado o salido del espacio aéreo de un **Aeropuerto** específico.

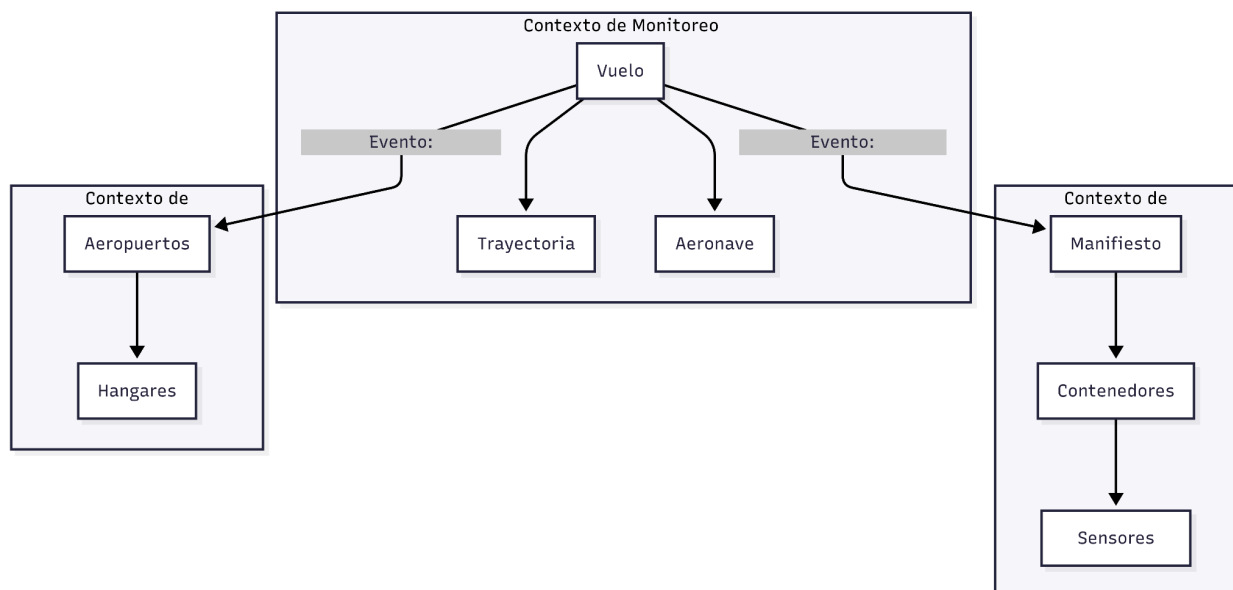
5.2.4. Eventos de Dominio

Sucesos relevantes que disparan acciones en otros módulos.

- **VueloDesviado:** Dispara una alerta en el sistema de logística para recalcular tiempos de entrega.
- **CargaEntregada:** Actualiza el estado del contenedor y libera espacio en los recursos de monitoreo activo.

5.2.5. Bounded Context (Contextos Delimitados)

Para evitar la ambigüedad, el sistema se divide en límites lógicos claros:

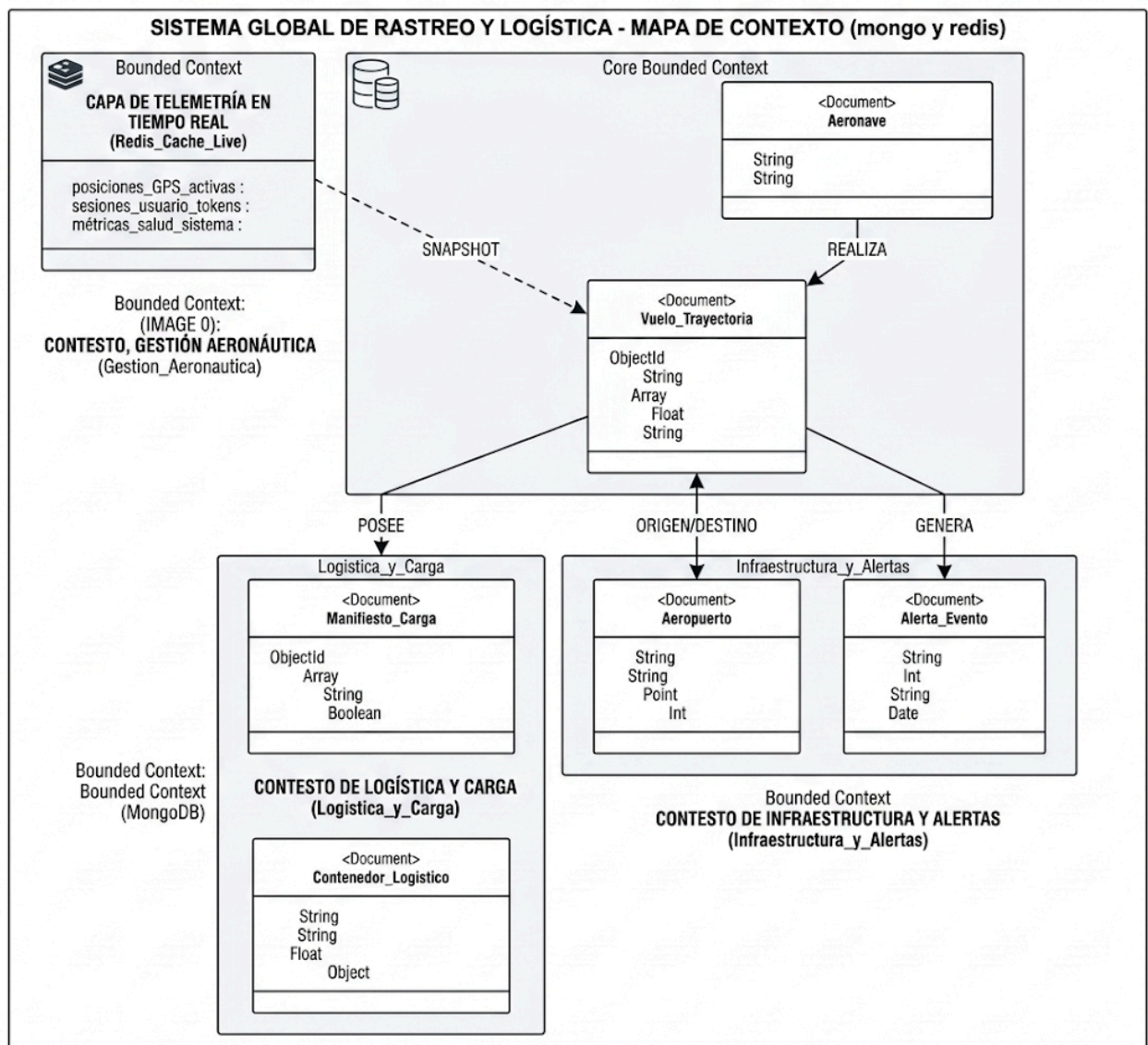


Análisis del Stack según el Diagrama

- **Redis:** Gestiona la carga transaccional de las **posiciones_GPS_activas**.
- **MongoDB:** Soporta la complejidad de los tipos de datos como **Point** (para Aeropuertos), **Array** (para trayectorias) y **Object** (para sensores IoT).
- **Integración:** La relación de **SNAPSHOT** es el puente crítico que permite que el sistema pase de un estado de "monitoreo vivo" a uno de "análisis logístico histórico".

6. Aplicación de la metodología al caso de estudio

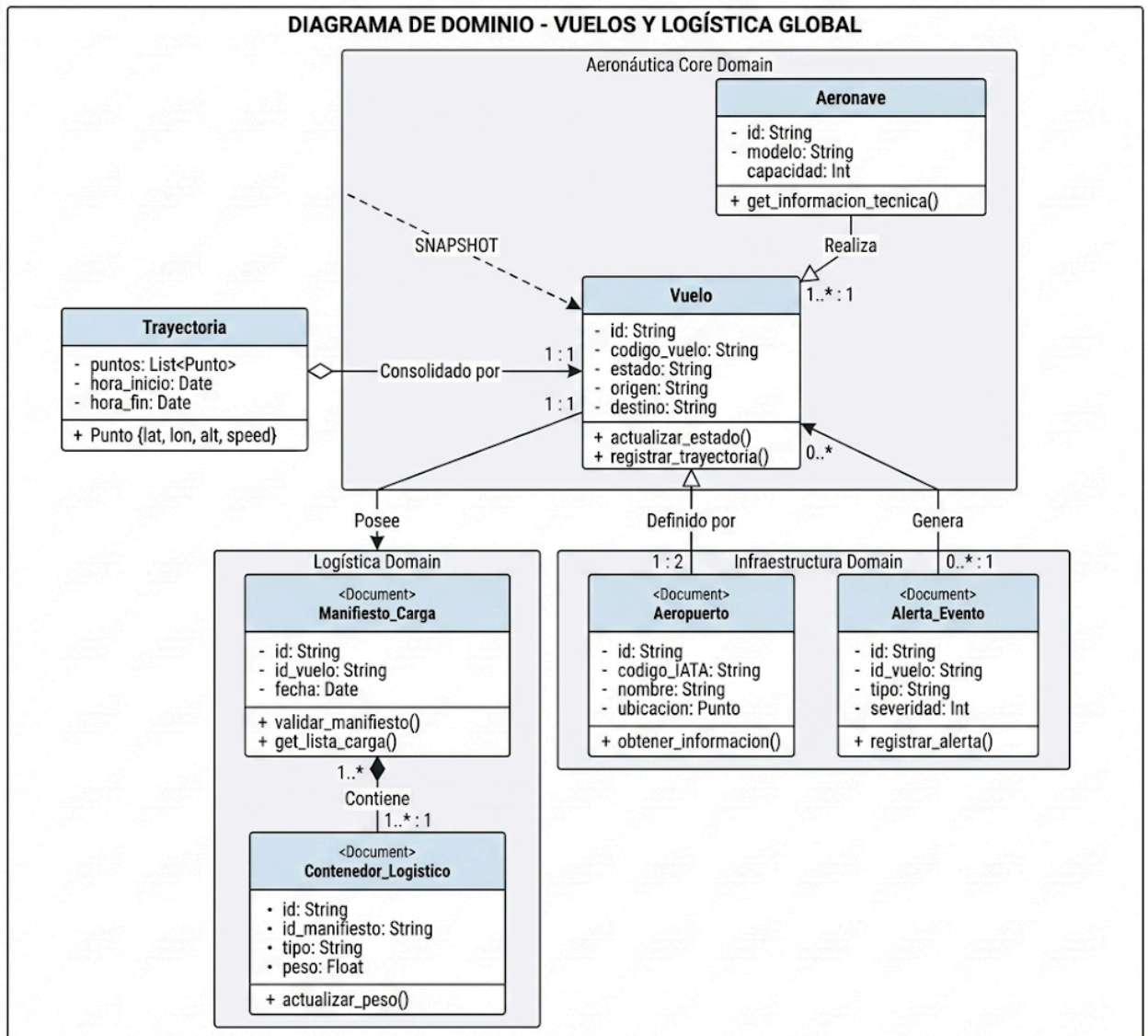
6.1 Mapa de contexto



El mapa representa la arquitectura estratégica del sistema mediante la metodología de Diseño Dirigido por el Dominio (DDD). Se visualiza la separación entre la Capa de Telemetría (gestionada en Redis para garantizar latencia mínima en datos volátiles de GPS y sesiones) y los contextos persistentes en MongoDB.

El núcleo del sistema (Gestión Aeronáutica) actúa como el integrador principal, transformando los snapshots de tiempo real en trayectorias históricas, las cuales se vinculan directamente con los dominios de Logística de Carga e Infraestructura. Esta organización asegura que el procesamiento masivo de datos geográficos no comprometa la integridad ni el rendimiento de los registros operativos y administrativos del proyecto.

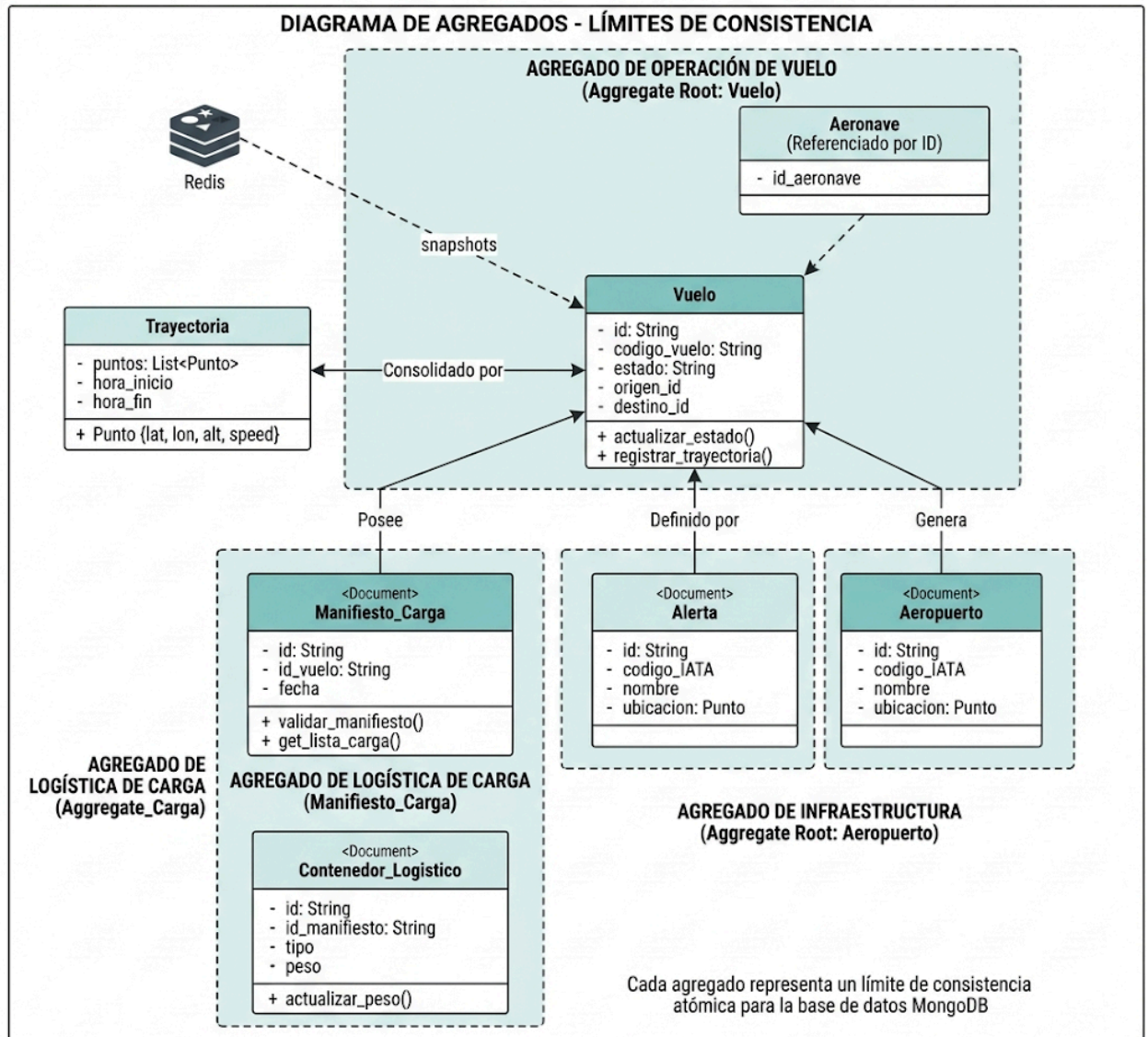
6.2 Diagrama de dominio



Este diagrama detalla la estructura lógica y las reglas de negocio que rigen el sistema. El Vuelo se establece como la entidad central, la cual consolida los datos de Trayectoria recibidos por la telemetría para definir el estado operativo de la misión.

El modelo refleja una jerarquía clara donde una Aeronave realiza múltiples vuelos, y cada uno de estos posee un Manifiesto de Carga que desglosa los Contenedores Logísticos transportados. Asimismo, el dominio integra la relación con la Infraestructura (Aeropuertos) y la capacidad autónoma de generar Alertas, permitiendo que el sistema reaccione ante desviaciones o incidentes de forma automatizada. Las relaciones de cardinalidad aseguran la trazabilidad completa del activo desde su origen hasta su destino final.

6.3 Diagrama de agregados



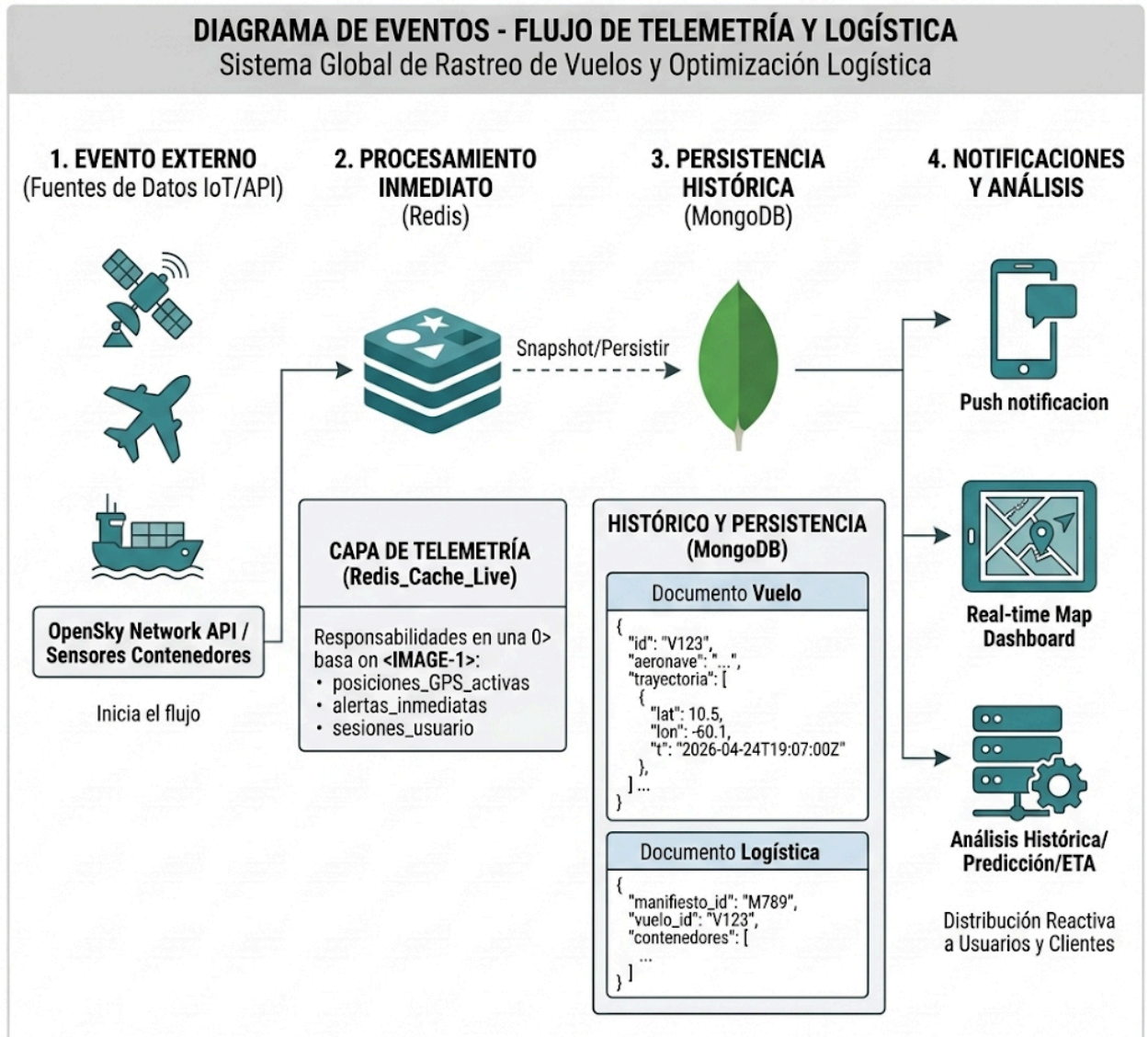
El diagrama define los Límites de Consistencia del sistema, agrupando las entidades en estructuras que garantizan la integridad de los datos en MongoDB. Cada agregado posee una Raíz de Agregado (Aggregate Root) que es la única puerta de entrada para modificar los datos internos:

- **Agregado de Operación de Vuelo:** Centraliza la lógica de la trayectoria. Es el componente crítico que recibe los snapshots de Redis y los consolida en un documento histórico de vuelo, asegurando que la ruta y el estado del avión sean siempre coherentes.
- **Agregado de Logística de Carga:** Protege la relación entre el manifiesto y sus contenedores. Garantiza que cualquier actualización en la carga (como cambios de peso o tipo) se valide contra el manifiesto global para evitar errores en el balance del vuelo.

- **Agregado de Infraestructura:** Actúa como un catálogo de referencia para aeropuertos y alertas, proporcionando la ubicación geoespacial exacta necesaria para los cálculos de proximidad y logística.

Este diseño permite que el sistema maneje ráfagas de datos en tiempo real sin sacrificar la estabilidad y el orden de la base de datos persistente.

6.4 Diagrama de eventos

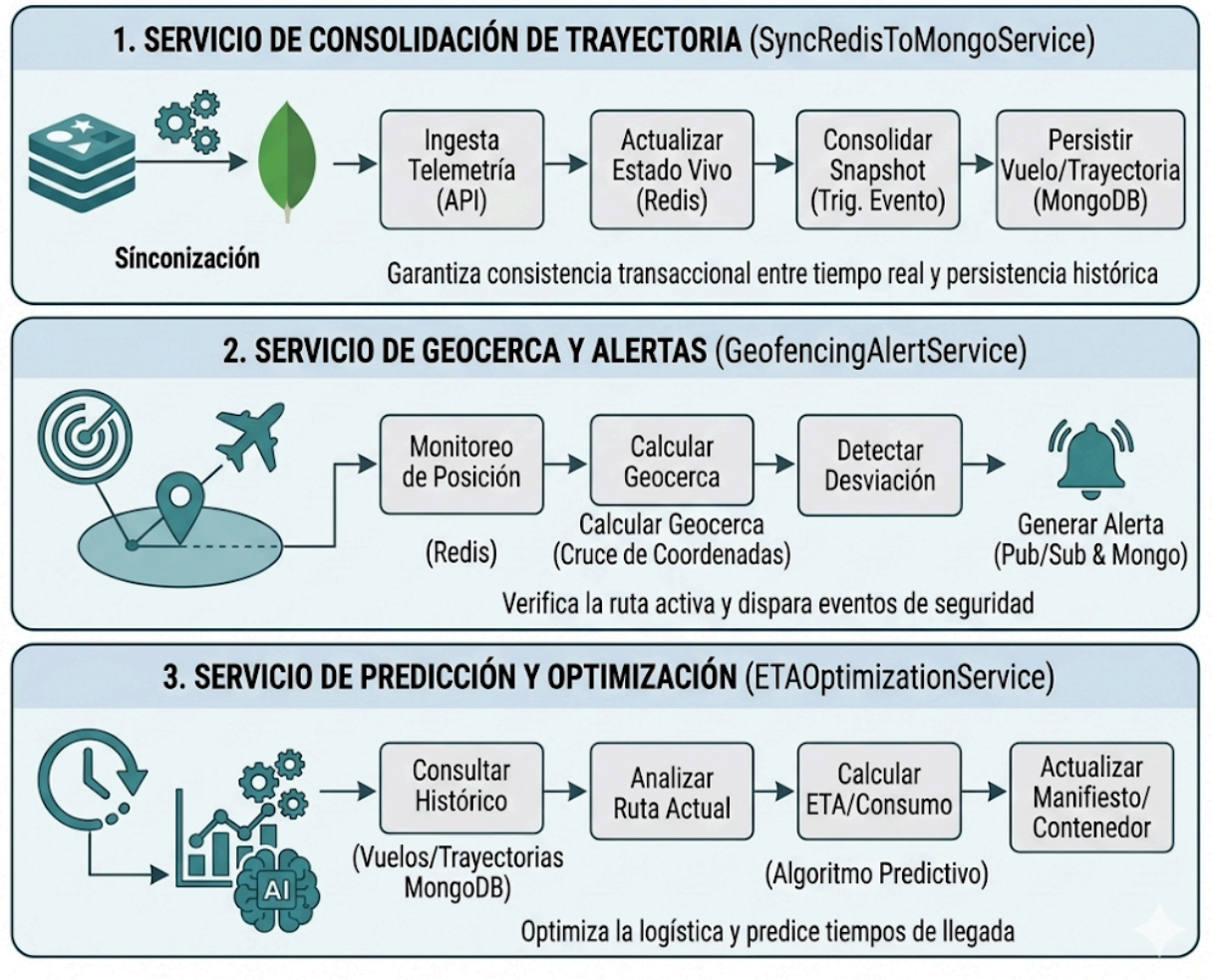


El diagrama ilustra el flujo reactivo de la información, basado en una arquitectura dirigida por eventos (Event-Driven Architecture). El ciclo de vida del dato se divide en cuatro fases críticas:

1. Ingesta (Evento Externo): Los datos brutos provenientes de APIs globales (como OpenSky) y sensores IoT de contenedores inician el flujo del sistema.
2. Procesamiento Inmediato (Redis): Los eventos de telemetría se capturan en la capa de Redis, permitiendo una respuesta instantánea para el rastreo activo y la gestión de sesiones con latencia mínima.
3. Persistencia Crítica (MongoDB): Mediante un proceso de Snapshot, los estados de los vuelos y la logística se consolidan en documentos BSON dentro de MongoDB, transformando eventos volátiles en registros históricos inmutables para auditoría.
4. Distribución y Análisis: Finalmente, el sistema propaga los eventos hacia los usuarios finales a través de notificaciones push, actualizaciones en tiempo real del Dashboard y servicios de análisis predictivo para la estimación de tiempos de llegada (ETA).

6.5 Servicios de dominio

SISTEMA GLOBAL DE RASTREO Y LOGÍSTICA
DIAGRAMA DE SERVICIOS DE DOMINIO - PROCESOS TRANSACCIONALES DE ALTO NIVEL

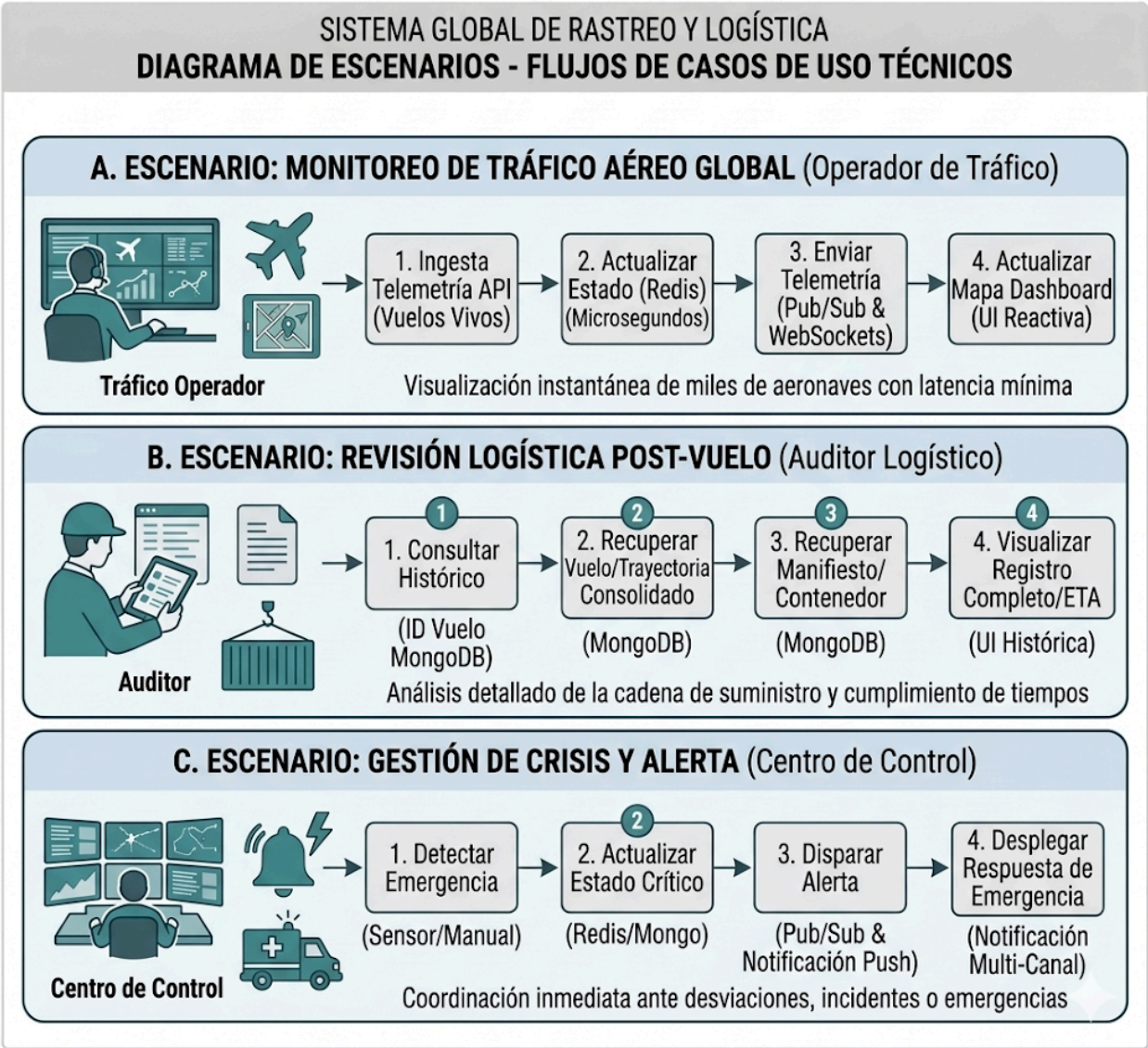


El diagrama expone los Servicios de Dominio, que representan las operaciones de lógica de negocio complejas que no pertenecen de forma natural a una sola entidad o agregado. Estos servicios actúan como coordinadores entre la infraestructura tecnológica y las reglas operativas:

- **SyncRedisToMongoService:** Orquestador de persistencia que garantiza la transición fluida de los datos desde la memoria volátil de Redis hacia la base de datos documental MongoDB, asegurando que ninguna métrica de vuelo se pierda al finalizar un trayecto.
- **GeofencingAlertService:** Servicio analítico que utiliza las capacidades geoespaciales para monitorear en tiempo real si una aeronave se desvía de su ruta predefinida, disparando alertas críticas mediante el modelo Pub/Sub.

- **ETAOptimizationService:** Motor de inteligencia logística que consulta los volúmenes masivos de datos históricos en MongoDB para calcular tiempos estimados de llegada y optimizar la cadena de suministro de los contenedores.

6.6 Diagrama de escenarios



Este diagrama detalla la ejecución práctica del sistema a través de tres flujos de usuario críticos (Casos de Uso), demostrando cómo interactúan los actores con la infraestructura técnica:

- **A. Monitoreo de Tráfico Global:** Describe el flujo de alta velocidad donde el Operador recibe actualizaciones en microsegundos. La telemetría fluye

desde las APIs externas hacia el Dashboard mediante WebSockets, utilizando Redis como motor de baja latencia.

- **B. Revisión Logística Post-Vuelo:** Enfocado en el Auditor, este escenario muestra la recuperación de datos históricos desde MongoDB. Permite analizar la trazabilidad completa del manifiesto de carga y la trayectoria consolidada una vez finalizada la operación.
- **C. Gestión de Crisis y Alerta:** Ilustra la respuesta reactiva del Centro de Control ante emergencias. Muestra la cadena de disparo desde la detección del sensor hasta la notificación multicanal, garantizando una coordinación inmediata ante incidentes.